

SIG PLATFORM

Audit de conformité au cahier des charges N °2, de sécurité OWASP Top 10 et de qualité logicielle : ce qui est fait, ce qui est partiel, ce qui reste à faire, et le plan d'action priorisé pour la mise en production.

GeoLight Service — SIG.tg SARL

Plateforme métier de cartographie et d'analyse spatiale · sig-platform (Django 6.1+ GeoDjango)

RÉFÉRENTIEL : CAHIER DES CHARGES N°2 — SIG PLATFORM

PÉRIMÈTRE : CODE · TESTS · SÉCURITÉ · RGPD · PERFORMANCE

18 SEPTEMBRE 2026

Table des matières

Synthèse exécutive	1
1. Contexte, objectifs et périmètre de l'audit	2
1.1 Contexte du projet	2
1.2 Périmètre et méthode de vérification	2
2. Cartographie technique de l'existant	3
2.1 Stack technique : prévu contre observé	3
2.2 Applications Django : 26 prévues contre 17 réelles	3
2.3 Modèles de données et qualité de la livraison	4
3. Conformité fonctionnelle au cahier des charges	5
3.1 Avancement par module et score par domaine	5
3.2 Écarts fonctionnels détaillés	6
3.3 Sécurité et conformité exigées (§ 5 du CDC)	7
3.4 Performance et indicateurs chiffrés (§ 6 et § 11 du CDC)	7
3.5 Les dix livrables contractuels (§ 8 du CDC)	8
4. Audit de sécurité — OWASP Top 10 et RGPD	9
4.1 Grille OWASP Top 10	9
4.2 A05 — La chaîne de service de production (constat critique C1)	10
4.3 Conformité RGPD	10
4.4 Points forts de sécurité à préserver	11
5. Qualité du code et tests	11
5.1 Points forts à capitaliser	11
5.2 Dette technique identifiée	11
5.3 Reproductibilité de l'environnement de test	12
6. Plan d'action priorisé	12
6.1 Vue d'ensemble des constats	12
6.2 Séquencement recommandé	14
7. Conclusion	14

Synthèse exécutive

Le présent rapport évalue la conformité du projet **SIG Platform** — l'espace métier sécurisé de l'écosystème SIG.tg — par rapport à son cahier des charges (document de référence « CAHIER DES CHARGES N°2 – SIG PLATFORM », 12 pages), et analyse l'état de sécurité et de qualité du code livré dans l'archive « sig-platform-audit.zip ». L'audit a porté sur l'intégralité du dépôt : dix-sept applications Django (Django 6.1 + GeoDjango), une configuration multi-environnements (dev, staging, production), une conteneurisation Docker (PostGIS 17-3.5, Redis 8, worker Celery), une API REST Django REST Framework et une suite de tests automatisés de 391 cas.

Le premier constat est nettement favorable : le socle technique est **remarquablement sain**, et sensiblement plus abouti que celui de la vitrine au moment de son premier audit. La suite de tests exécutée par l'audit dans un environnement propre affiche **391 tests réussis sur 391** en 81 secondes, avec une couverture mesurée d'environ 80 % — la cible « > 80 % » du cahier des charges est atteinte à la limite. L'audit de configuration de production « check --deploy » ne remonte **aucun avertissement** (contre six à l'époque du premier audit de la vitrine). Le multi-tenant est appliqué systématiquement, les rôles Admin / Éditeur / Visualisateur sont contrôlés côté serveur sur chaque vue d'écriture, un journal d'audit complet est alimenté, les quotas d'abonnement sont verrouillés serveur, et une politique de sécurité des contenus (CSP) stricte sans aucun CDN tiers complète le dispositif.

En revanche, l'audit identifie **17 constats** (2 critiques, 8 majeurs, 7 mineurs) et estime la conformité globale au cahier des charges à environ **68 %**. Six des neuf modules fonctionnels décrits sont livrés de bout en bout ; les écarts les plus structurants portent d'une part sur la chaîne de mise en production (le conteneur applicatif lance le serveur de développement « runserver », sans Gunicorn, Nginx ni planificateur Celery Beat, et aucune sauvegarde n'est outillée), et d'autre part sur le module Analyses spatiales, cœur de la proposition de valeur : sur les cinq outils exigés, seuls deux (zone tampon, intersection/découpe) sont conformes — l'isochrone, le calcul de surfaces et périmètres et la requête spatiale sont absents. Les intégrations externes exigées (paiement Stripe/Flutterwave, stockage S3/OVH, SMTP transactionnel, Sentry, Prometheus/Grafana) ne sont pas encore intégrées, ce qui est cohérent avec une phase de développement mais doit être planifié.

391/391 tests automatisés réussis	0 alerte check --deploy (prod)	17 constats (2 C / 8 M / 7 m)	≈ 68 % conformité au CDC N°2
---	--	---	--

Chiffres clés de l'audit — exécution des tests et vérifications du 18 septembre 2026

Le plan d'action proposé chiffre l'effort de remise à niveau entre **29 et 41 jours-homme** (environ sept à neuf semaines pour un développeur confirmé), ce qui reste compatible avec les phases « tests et recette » (3 semaines) et « déploiement et formation » (1 semaine) prévues au planning des 26 semaines du cahier des charges. Les deux actions critiques (chaîne de production, outils d'analyse manquants) conditionnent la recette ; les huit actions majeures couvrent la monétisation, l'API, la performance, les notifications et la documentation exigée.

1. Contexte, objectifs et périmètre de l'audit

1.1 Contexte du projet

L'offre SIG.tg se compose de deux entités complémentaires : la vitrine publique **SIG Web Site** (auditée séparément le 16 septembre 2026), chargée de présenter la plateforme et de capturer les inscriptions, et l'espace métier **SIG Platform**, accessible uniquement aux utilisateurs authentifiés, où les organisations importent, visualisent, analysent et exportent leurs données géographiques. Le flux produit attendu relie les deux : un visiteur s'inscrit depuis la vitrine (« subscribe to the service »), l'administrateur reçoit l'inscription et crée le compte, et l'utilisateur accède alors à la plateforme pour y travailler. Le cahier des charges N°2 décrit une plateforme bâtie sur Django 5.2+ avec GeoDjango, PostgreSQL/PostGIS, Redis et Celery, conteneurisée via Docker et destinée à un VPS Contabo (Ubuntu 22.04, Nginx, Unicorn, sauvegardes quotidiennes), pour un budget estimé de **63 000 € HT** et un planning de **26 semaines** en six sprints, dix livrables contractuels et dix indicateurs de performance dont « zéro faille critique » au score de sécurité.

1.2 Périmètre et méthode de vérification

Sont entrés dans le périmètre : le code source complet de « sig-platform-audit.zip » (18 septembre 2026 — 17 applications, configuration, gabarits, JavaScript, fichiers Docker, documentation), le cahier des charges N°2, l'exécution de la suite de tests, la mesure de couverture et l'audit de configuration de production. L'historique Git est fourni anonymisé (auteurs et dates neutralisés) : l'appréciation de l'avancement s'appuie donc exclusivement sur l'état du code, non sur sa chronologie. Sont hors périmètre : l'infrastructure d'hébergement réelle (VPS, domaine), les intégrations externes vivantes (Stripe, SendGrid, Sentry, stockage objet) et l'exploitation de production.

La méthode combine cinq techniques reproductibles. Premièrement, une revue manuelle approfondie du code croisée avec chaque exigence du cahier des charges. Deuxièmement, l'exécution de la suite complète de tests dans un environnement Python propre avec les dépendances du requirements (Django 6.1.1), sur base SpatiaLite — faute de PostgreSQL/PostGIS disponible sur le poste d'audit, l'option « SIG_DB_ENGINE=sqlite » prévue par le projet elle-même a été utilisée, ce qui garantit des tests rejouables mais peut masquer des spécificités PostGIS. Troisièmement, l'audit natif « manage.py check --deploy » sous le module de réglages de production (clé de longueur suffisante, ALLOWED_HOSTS défini, HTTPS_ONLY activé). Quatrièmement, la mesure de couverture avec coverage.py (3 303 instructions exécutables, hors migrations et tests). Cinquièmement, des contrôles ciblés : cohérence des migrations, quotas, permissions, conteneurisation et intégrations. Chaque constat est rattaché à un fichier et priorisé par gravité (Critique / Majeur / Mineur).

2. Cartographie technique de l'existant

2.1 Stack technique : prévu contre observé

La stack réellement mise en œuvre est globalement conforme — et parfois supérieure — aux choix imposés : Django 6.1 avec GeoDjango dépasse l'exigence « 5.2+ », la base spatiale s'appuie sur PostGIS 17-3.5 dans Docker avec bascule Spatialite prévue pour le développement local, et le front-end respecte la consigne « JavaScript vanilla + Leaflet » tout en auto-hébergeant la bibliothèque et les polices, ce qui permet une CSP stricte sans CDN tiers. Les écarts se concentrent sur la couche d'exploitation : le conteneur applicatif lance le serveur de développement au lieu de Gunicorn, aucun cache Redis n'est configuré (Redis n'est que le broker Celery), et le stockage objet S3/OVH exigé est absent.

Tableau 1 — Stack technique : prévisions du cahier des charges contre réalité du code

Composant	Prévu (CDC § 4.1)	Observé dans le code	Verdict
Framework	Django 5.2+ avec GeoDjango	Django 6.1 + GeoDjango (GeometryField, SRID 4326)	OK (supérieur)
Base de données	PostgreSQL 16+ / PostGIS 3.0+	Image postgres/postgis:17-3.5, moteur postgres, 32 migrations cohérentes	OK
API REST	DRF + djangorestframework-gis (GeoJSON)	DRF 3.16, API lecture seule org-scopée ; djangorestframework-gis absent, aucune géométrie exposée	Partiel
Cache / queue	Redis 6+ : cache tuiles + broker Celery	Redis 8 broker uniquement ; CACHES non configuré, aucun cache de tuiles	Partiel
Tâches asynchrones	Celery + Celery Beat	Celery 5.6 complet (JSON, limites, planning Beat) ; Beat absent du docker-compose	Partiel
Front-end	HTML5/CSS3/JS vanilla + Mapbox GL ou Leaflet	Conforme : Leaflet auto-hébergé, JS vanilla modulaire, CSP stricte	OK
Conteneurisation	Docker + Docker Compose	Dockerfile (GDAL/GEOS/PROJ) + compose (db, redis, web, celery) ; mais « runserver » — ni Gunicorn ni Nginx ni Beat	Partiel
Stockage des fichiers	S3-compatible (OVH Object Storage)	Stockage local MEDIA_ROOT ; aucune dépendance S3/MinIO	Manquant

Comparaison de la stack prévue et de la stack observée (source : requirements/, config/settings/, docker/)

2.2 Applications Django : 26 prévues contre 17 réelles

Le cahier des charges organise le code en **26 applications** réparties par domaine. Le dépôt en contient **dix-sept**, toutes documentées et testées. La comparaison montre que l'essentiel de la différence relève d'une consolidation assumée plutôt que d'un manque fonctionnel : les utilitaires, la gestion des fichiers, les tâches asynchrones ou les e-mails sont réalisés sans application dédiée. Deux absences sont toutefois réelles : l'application **webhooks** (exigée en Sprint 6) n'a aucun équivalent fonctionnel dans le code, et le **cache** n'est implémenté nulle part.

Tableau 2 — Correspondance entre les 26 applications prévues et le dépôt livré

Domaine CDC	Applications prévues	Réalisation observée	Verdict
Socle	core, common, files, cache, tasks, audit, email, search	core, audit, search présents ; common/files consolidés (utils, FileField + services) ; email = django.core.mail direct ; cache absent	Partiel
Gestion utilisateur	accounts, organizations, teams, subscriptions, preferences	5/5 présentes et conformes (User rôlié, Organization, Invitations, Plan/Invoice/quotas, UserPreference)	OK (5/5)
Géospatial	maps, layers, data_import, spatial_analysis, exports, projects	6/6 présentes ; spatial_analysis limité à 4 outils sur 5 exigés	Partiel
Intégrations	osm_integration, api, webhooks	api présente (DRF) ; osm_integration réduite aux fonds de tuiles du viewer ; webhooks totalement absente	Manquant
Interface	dashboard, notifications	2/2 présentes ; notifications non branchée sur les événements métier (cf. M5)	Partiel
Tests & données	tests, fixtures	391 tests répartis par application ; jeu de démonstration via la commande seed_demo (388 lignes, données Lomé)	OK (fonctionnel)

Couverture applicative : 17/26 en découpage strict (65 %) ; les domaines fonctionnels sont pour leur part couverts

2.3 Modèles de données et qualité de la livraison

Les dix modèles principaux listés au cahier des charges existent tous, avec des noms parfois différents mais des rôles équivalents : Organization, User rôlié, Map (statut, fond, centre, zoom, couches via MapLayer), Layer et Feature (géométrie SRID 4326, propriétés JSON), Project (statut, progression, membres), ImportJob, AnalysisJob, ExportJob et Plan/Invoice pour l'abonnement. Trois nuances méritent une décision produit documentée : l'entité Team est réalisée par des invitations rôliées (token aléatoire de 48 caractères, expiration 7 jours) plutôt qu'un objet « équipe » ; l'entité Subscription est portée par le champ plan de l'Organization augmenté d'Invoice ; l'Organization prévoyait des champs e-mail, téléphone et adresse absents. Les index, contraintes d'unicité et relations sont soignés.

La livraison est propre et professionnelle, en net progrès par rapport à l'archive de la vitrine : aucun environnement virtuel, aucune base et aucun fichier .env avec secret réel dans l'archive ; un .env.example complet documente toutes les variables attendues. Les fichiers de dépendances sont séparés par environnement, bornés par versions et sans doublon — il manque toutefois un fichier de verrouillage et une analyse pip-audit. Le README (25 Ko) couvre l'installation, PostgreSQL/PostGIS, Redis, les variables, les migrations, Celery et les tests. Enfin, le cahier des charges annonce « dix modules principaux » mais n'en décrit que neuf (§ 3.1 à 3.9) : cette incohérence documentaire, sans conséquence technique, gagnera à être tranchée avec le donneur d'ordre.

3. Conformité fonctionnelle au cahier des charges

3.1 Avancement par module et score par domaine

La figure 1 présente le taux de couverture des exigences pour chacun des neuf modules décrits : six modules sont livrés de bout en bout ou presque, deux sont partiellement conformes (analyses spatiales : 40 % ; paramètres : 60 %) et un est presque complet (exports : la projection exigée manque). La figure 2 décompose la conformité globale — estimée à ≈ 68 % — par domaine, selon une pondération assumée : modules fonctionnels 30 %, architecture et sécurité 20 % chacune, découpage applicatif, intégrations externes et livrables 10 % chacun.

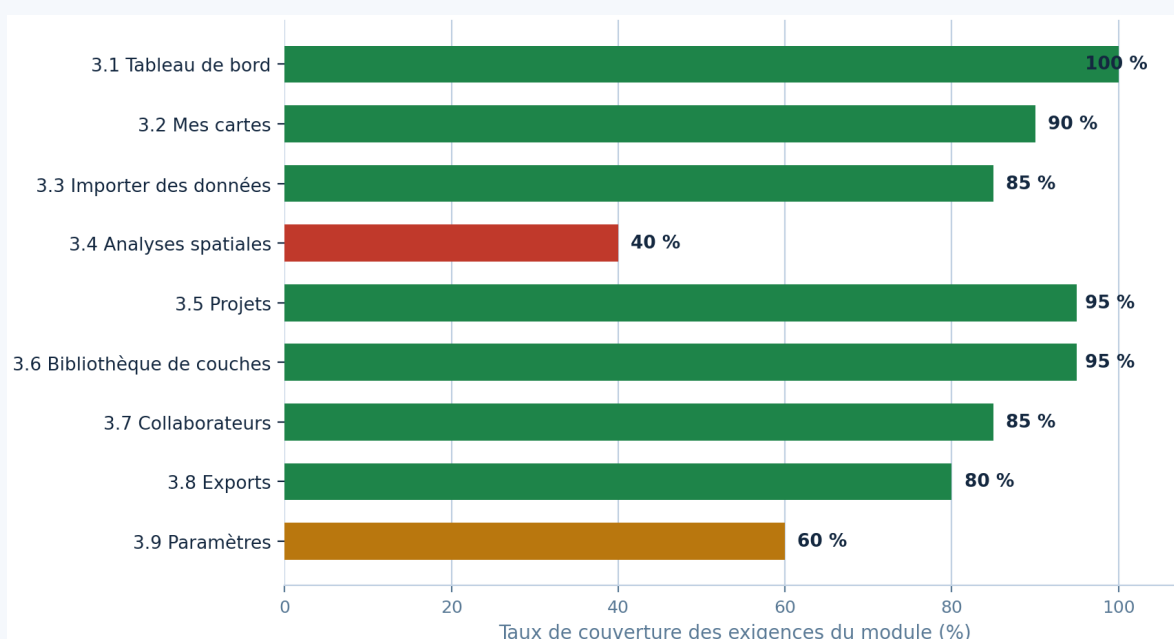


Figure 1 — Taux de couverture des exigences par module du cahier des charges (estimation d'audit)

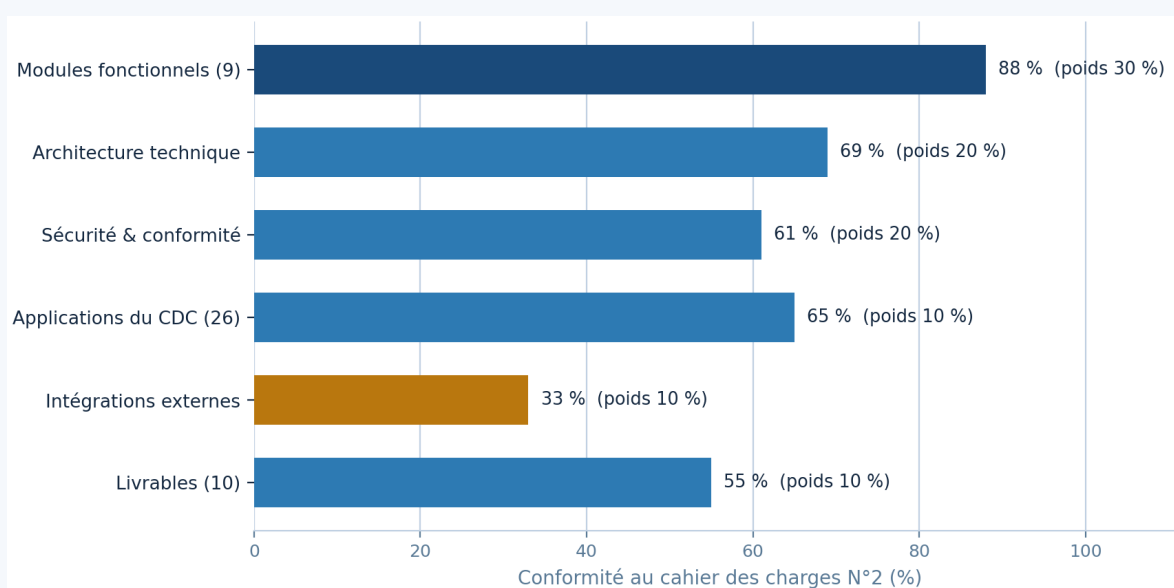


Figure 2 — Conformité par domaine et pondération retenue pour le score global (≈ 68 %)

Tableau 3 — Les neuf modules fonctionnels du cahier des charges : état observé

Module (CDC)	Statut	État observé dans le code
3.1 Tableau de bord	Terminé	KPIs (cartes, entités, stockage, collaborateurs), cartes récentes, flux d'activité 14 jours (SVG), actions rapides
3.2 Mes cartes	Terminé	Grille paginée, recherche, filtre statut, tri, aperçus, éditeur Leaflet complet (couches, ordre, visibilité, opacité), duplication, suppression ; l'action « partager » exigée est absente
3.3 Importer des données	Terminé	Glisser-déposer 4 formats (CSV, GeoJSON, Shapefile ZIP, KML), validation, quotas, tâche Celery, historique paginé ; aperçu des premières lignes et case « remplacer existant » absents
3.4 Analyses spatiales	Partiel	Buffer, intersection, différence et bounding box conformes (reprojection UTM, réparation de géométries, résultat en couche) ; isochrone, surfaces/périmètres et requête spatiale absents (cf. 3.2.1)
3.5 Projets	Terminé	CRUD complet, statuts, progression %, membres, échéance, pagination, recherche et filtres
3.6 Bibliothèque de couches	Terminé	Liste avec filtres type/géométrie, recherche, métadonnées, détail cartographique, endpoint GeoJSON, suppression
3.7 Collaborateurs	Terminé	Liste des membres rôlisée, invitation par e-mail (token 48 c., expiration 7 j), changement de rôle, retrait, quotas ; statut en ligne/hors ligne absent
3.8 Exports	Partiel	6 formats conformes (GeoJSON, CSV, Shapefile ZIP, KML, carte PDF, carte PNG), Celery, historique, téléchargement ; choix de projection et options avancées absents
3.9 Paramètres	Partiel	Profil ✓, Préférences ✓ (unités, coordonnées, fond, langue), Abonnement ✓ (plan, usage, factures) ; Sécurité partiel (sessions actives absentes) ; onglet Notifications absent ; suppression de compte absente

État des neuf modules décrits au § 3 du cahier des charges — constats sourcés (vues, modèles, gabarits, tests)

3.2 Écarts fonctionnels détaillés

3.2.1 — Trois des cinq outils d'analyse spatiale exigés sont absents

Le cahier des charges décrit cinq outils : zone tampon (buffer), zone de chalandise (isochrone), intersection/clip, calcul de surfaces et périmètres, et requête spatiale (intersecte, contient, est dans). Le dépôt implémente quatre types : buffer, intersect, difference et bounding box — soit **deux outils conformes et deux bonus non demandés**. Manquent l'isochrone, le calcul de surfaces/périmètres et la requête spatiale. L'existant est de bonne facture : distances métriques via reprojection UTM calculée depuis l'étendue des données, géométries invalides réparées avant traitement, résultat versé dans une couche réutilisable, exécution en tâche Celery avec suivi. L'absence la plus structurante est l'**isochrone** : elle exige un moteur de réseau routier (pgRouting sur PostGIS, ou service OSRM/Valhalla) qui ne figure ni dans les dépendances ni dans le compose — c'est un chantier d'architecture, pas un simple ajout. Le calcul de surfaces/périmètres et la requête spatiale s'appuient directement sur les primitives GEOS/PostGIS déjà utilisées et peuvent être livrés rapidement.

3.2.2 — Paramètres : deux onglets sur cinq incomplets ou absents

L'onglet « Sécurité » permet de changer le mot de passe (avec re-validation de session, bonne pratique), mais n'offre ni la consultation des sessions actives ni la déconnexion à distance exigées — Django conserve pourtant toutes les briques (sessions en base, listage, suppression). L'onglet « Notifications » exigé (activation/désactivation des e-mails, push, alertes de projets, partages de cartes, alertes système) n'existe pas : aucune préférence de notification n'est modélisée, et le centre de notifications en place est un flux read/unread sans préférences. Enfin, la « zone de danger » avec suppression de compte est absente du module Abonnement. Les autres onglets (Profil, Préférences, Abonnement) sont conformes.

3.2.3 — Exports et cartes : nuances de conformité

Le module Exports est fonctionnellement riche : six formats conformes (dont les feuilles de carte PDF générées avec ReportLab et les instantanés PNG), exécution asynchrone, historique avec téléchargement et suppression. Il manque le choix de projection (WGS 84, Web Mercator, UTM) — les données restent en SRID 4326 sauf re-projection ponctuelle à l'analyse — et les options avancées (attributs, légende, qualité/résolution, filtre spatial). Côté cartes, les statuts exigés « public / privé / partagé / brouillon » sont remplacés par « brouillon / publié / archivé » (choix cohérent avec un espace authentifié, mais divergent du CDC), et l'action « partager » n'existe pas. Ces divergences sont acceptables si elles sont assumées contractuellement, mais doivent être listées explicitement dans une note de décision produit.

3.3 Sécurité et conformité exigées (§ 5 du CDC)

Tableau 4 — Les huit mécanismes de sécurité exigés : état observé

Exigence (CDC § 5)	Statut	Détail
Chiffrement des échanges (HTTPS / TLS 1.3)	Terminé	Réglages complets derrière HTTPS_ONLY : redirection SSL, HSTS un an (sous-domaines + preload), cookies Secure/HttpOnly/SameSite ; TLS 1.3 relève du serveur web (Nginx absent, cf. C1)
Contrôle d'accès RBAC (3 rôles)	Terminé	Rôles contrôlés serveur par decorators (editor_required, org_admin_required) sur chaque vue d'écriture ; refus par PermissionDenied (403)
Isolation multi-tenant	Terminé	Chaque queryset filtré par l'organisation (middleware request.organization) ; l'API DRF applique le même scoping ; aucun accès croisé constaté
Journalisation des actions critiques	Terminé	AuditLog (action, utilisateur, organisation, objet, horodatage) alimenté en 26 points du code ; panneau interne /panel/ réservé staff
Mots de passe : 8 caractères + 4 classes	Partiel	Validateurs Django par défaut (longueur 8, mots courants, tout-numérique) ; aucune exigence de classes de caractères
2FA en option	Manquant	Aucun mécanisme (TOTP, SMS) dans le code
Sauvegardes automatisées quotidiennes	Manquant	Aucune procédure, aucun volume de sauvegarde, aucune tâche planifiée
Conformité RGPD (droits d'accès, rectification, effacement, portabilité)	Partiel	Accès/rectification via le profil ; portabilité géographique via Exports ; effacement complet du compte absent ; minimisation respectée (aucune IP stockée)

Mécanismes de sécurité exigés par le § 5 du cahier des charges et constats associés

3.4 Performance et indicateurs chiffrés (§ 6 et § 11 du CDC)

Tableau 5 — Indicateurs cibles du cahier des charges et capacité actuelle

Indicateur (CDC)	Cible	État constaté
Chargement du dashboard	< 2 s	Socle favorable (annotations, select_related, pagination) ; non mesuré — mesure à réaliser en recette
Affichage 100 000 entités	< 1,5 s (MVT + cache Redis)	Non atteignable en l'état : aucun ST_AsMVT, aucun cache ; GeoJSON en streaming sans pagination ni cache
Import de 10 000 entités	< 5 s	Non mesuré — parsing streaming et création en lot présents ; à benchmarker après migration PostGIS
Analyse buffer/intersection	< 3 s (10 000 entités)	Non mesuré — GEOS avec reprojection UTM ; coût réel à évaluer
Utilisateurs simultanés	500	Non testé — et le conteneur lance runserver (mono-processus) : Unicorn requis avant tout test de charge
Temps de réponse API moyen	< 200 ms	Non mesuré — API lecture seule paginée, throttling 600/min par utilisateur
Disponibilité (uptime)	99,9 %	Non garantissable — aucun monitoring, aucune alerte, aucune redondance outillée
Score de sécurité (audit)	Zéro faille critique	Atteint sur le plan applicatif : check --deploy 0 alerte, aucune faille critique identifiée ; la chaîne de service reste à corriger avant production

Indicateurs du § 11 du cahier des charges — état de capacité, hors mesures de charge à réaliser

3.5 Les dix livrables contractuels (§ 8 du CDC)

Tableau 6 — Suivi des dix livrables exigés

Livable exigé	Statut	Constat
1. Code source sur dépôt Git, branches dev/prod	Terminé	Dépôt structuré et propre ; l'archive étant anonymisée, la séparation des branches n'est pas vérifiable — à attester sur le dépôt réel
2. Documentation technique	Terminé	README exhaustif (installation, Docker, PostGIS, Redis, variables, migrations, Celery, tests) ; guide de déploiement Contabo à rédiger
3. Documentation administrateur	Partiel	Superuser et gestion courante couverts ; pas de guide dédié (organisations, plans, factures, panel /panel/)
4. Documentation API (Swagger / ReDoc)	Manquant	Aucun schéma OpenAPI : drf-spectacular absent, aucune route de documentation
5. Schéma de base de données (migrations + fixtures)	Terminé	32 migrations cohérentes (makemigrations --check : rien à faire) ; seed_demo : plans, organisation, membres, couches, cartes, projets, factures, notifications
6. Tests, couverture > 80 %	Terminé (limite)	391 tests réussis en 81 s (≈ 30 fichiers) ; couverture mesurée ≈ 80 % — pile sur la cible
7. Jeu de données de démonstration	Terminé	seed_demo.py (388 lignes) : données réalistes autour de Lomé, mots de passe jamais codés en dur
8. Plateforme en production (app.sigtech)	Non applicable	Conforme au planning : la mise en production suit la recette ; la chaîne de service exigée reste à construire (cf. C1)
9. Procédure de sauvegarde et de restauration	Manquant	Aucune documentation, aucun script, aucun test de restauration
10. Formation utilisateur (PDF ou vidéo)	Manquant	Aucun support de formation dans le dépôt

Suivi des dix livrables contractuels — état constaté au 18 septembre 2026

4. Audit de sécurité — OWASP Top 10 et RGPD

4.1 Grille OWASP Top 10

Le bilan de sécurité est le point fort de cet audit : pour la première fois sur l'écosystème SIG.tg, aucune catégorie OWASP n'est classée « Manquant ». Les fondations Django sont exploitées de manière homogène — ORM exclusif, CSRF actif, hachage robuste, secrets hors code — et des mécanismes rarement vus à ce stade sont déjà en place : contrôle de rôles côté serveur systématique, isolation multi-tenant appliquée à chaque requête, journal d'audit applicatif, quotas incompressibles, throttling de connexion et d'inscription, CSP stricte sans CDN tiers.

Tableau 7 — Couverture OWASP Top 10 (2021)

Catégorie OWASP	Verdict	Constats principaux
A01 — Contrôle d'accès défaillant	Bon	RBAC serveur systématique (403 sur toute écriture hors rôle) ; querysets scopés par organisation partout, y compris API ; réserve : throttling de connexion en cache local (par processus) — à migrer sur Redis partagé
A02 — Défaillances cryptographiques	Bon	HSTS, redirection SSL, cookies sécurisés (derrière HTTPS_ONLY) ; hachage pbkdf2 ; secrets hors code ; réserve : SECRET_KEY retombe sur « django-insecure-change-me » si la variable est oubliée
A03 — Injection	Bon	ORM exclusif (aucune requête brute) ; validation d'extension et de format à l'import ; échappement automatique des gabarits
A04 — Conception non sécurisée	Bon	Throttling connexion (10/5 min) et inscription (10/h) ; quotas serveur incompressibles ; invitations expirantes à token aléatoire ; redirection « next » validée
A05 — Mauvaise configuration	Partiel	check --deploy : 0 avertissement sous HTTPS_ONLY=1 ; mais durcissement conditionné par une variable, runserver en CMD, Rosetta monté sans condition d'environnement (accès staff seulement)
A06 — Composants vulnérables	Partiel	Django 6.1, DRF 3.16, Celery 5.6 récents et bornés ; pas de lock ni de pip-audit intégrés à la chaîne
A07 — Défaillances d'authentification	Partiel	Pas d'exigence de classes de caractères ; 2FA absente ; réinitialisation de mot de passe absente (M4) ; sessions actives absentes
A08 — Défaillances d'intégrité logicielle	Bon	Zéro ressource tierce en ligne : Leaflet, polices et icônes auto-hébergés ; CSP stricte avec relâchement contrôlé (admin, API navigateur)
A09 — Journalisation et surveillance	Partiel	Journal d'audit applicatif riche (26 points d'appel) et panneau staff ; mais aucune configuration LOGGING, pas de Sentry
A10 — SSRF	Bon	Aucune récupération d'URL fournie par l'utilisateur côté serveur ; fonds de carte = constantes de configuration

Grille OWASP Top 10 appliquée au dépôt sig-platform

4.2 A05 — La chaîne de service de production (constat critique C1)

Le Dockerfile se termine par **CMD** `["python", "manage.py", "runserver", "0.0.0.0:8000"]` et le docker-compose lance la même commande : or « runserver » est un serveur de développement, mono-processus, non durci, explicitement déconseillé par Django pour la production. Le cahier des charges exige pourtant Nginx en reverse proxy et Gunicorn en serveur WSGI (§ 4.2). De plus, le service Celery Beat est absent du compose alors que le planning `CELERY_BEAT_SCHEDULE` (purge des jobs bloqués toutes les 15 minutes) ne s'exécute jamais sans lui : un import ou un export interrompu resterait bloqué « `PROCESSING` » pour toujours, laissant l'utilisateur sans réponse et avec son quota consommé. Enfin, aucune sauvegarde n'est outillée. Lancée en l'état, la plateforme serait lente, sans planificateur et irrécupérable en cas d'incident de données. Le correctif doit précéder toute recette : Gunicorn en CMD, service Beat, Nginx, et `SECRET_KEY` rendue obligatoire.

Correctif proposé — Dockerfile (extrait), compose et réglages

```
# Dockerfile – produire avec un serveur WSGI, pas runserver
RUN pip install -r requirements/base.txt gunicorn
CMD ["gunicorn", "config.wsgi:application", "--bind", "0.0.0.0:8000",
    "--workers", "4", "--timeout", "120"]

# docker-compose.yml – ajouter le service planificateur
celery-beat:
  build: { context: .., dockerfile: docker/Dockerfile }
  command: celery -A config.celery beat --loglevel=info

# config/settings/base.py – rendre la clé obligatoire en prod
SECRET_KEY = config("SECRET_KEY") # plus de repli par défaut
```

Code 1 — Gunicorn + Celery Beat + clé secrète obligatoire (extraits indicatifs)

4.3 Conformité RGPD

Le traitement des données personnelles est globalement maîtrisé : le journal d'audit ne stocke aucune adresse IP, l'inscription collecte le strict nécessaire (nom, e-mail, organisation), les invitations expirent, et les données de démonstration sont fictives (lieux publics de Lomé). Sur les droits des personnes, l'accès et la rectification sont couverts par le profil ; la portabilité est partiellement servie par le module Exports (données géographiques, pas les données de compte) ; l'effacement complet du compte est absent — la suppression d'un utilisateur déclenche une anonymisation par détachement (`on_delete=SET_NULL`) plutôt qu'un effacement, choix défendable pour préserver la traçabilité mais à documenter dans une politique de confidentialité dédiée à la plateforme (celle de la vitrine ne couvre pas cet espace). Trois compléments sont recommandés : un parcours de suppression de compte dans la « zone de danger », une mention d'information à la création de compte, et une expiration des fichiers d'export prévue au cahier des charges mais absente du modèle.

4.4 Points forts de sécurité à préserver

L'audit souligne les acquis à conserver tels quels : l'isolation multi-tenant sans exception (aucun querset global non filtré constaté), les decorators de rôle refusant les écritures par 403 plutôt que par masquage d'interface, le journal d'audit alimenté par un helper centralisé résistant aux threads (nettoyage du thread-local en finally), les quotas serveur vérifiés avant toute création (cartes, couches, membres, stockage), le throttling de l'API (60/min anonyme, 600/min authentifié), la CSP stricte avec auto-hébergement intégral, la commande de démonstration qui génère des mots de passe aléatoires plutôt que de les coder en dur, et la tâche de récolte des jobs abandonnés — qui ne demandera qu'à être planifiée par un service Beat effectivement lancé.

5. Qualité du code et tests

5.1 Points forts à capitaliser

La qualité du code est le premier atout du projet. La suite de tests, exécutée par l'audit dans un environnement propre, compte **391 tests tous réussis en 81 secondes**, couvrant les modèles, les vues, les services d'import et d'analyse, les tâches Celery, les permissions et les middleware — soit environ 30 fichiers de test pour 3 303 instructions exécutables, avec une couverture mesurée à **79,6 %, pile sur la cible « > 80 % » du cahier des charges**. L'architecture est homogène et lisible : séparation nette entre modèles, services (parsing d'import, géotraitement, génération d'exports) et tâches asynchrones ; annotations de requête anti-N+1 explicitement commentées ; pagination systématique ; streaming du GeoJSON pour les couches volumineuses ; en-têtes de modules qui documentent les choix techniques. L'internationalisation est réelle : langue par défaut anglaise, bascule française, fichiers .po/.mo compilés, Rosetta réservé au staff, contenus bilingues via django-modeltranslation. Le JavaScript vanilla est structuré et compatible avec la CSP stricte — aucun script en ligne.

5.2 Dette technique identifiée

- **Chaîne de service** : runserver en CMD Docker, Unicorn/Nginx absents, service Celery Beat absent du compose (cf. C1) ;
- **Cache** : CACHES non configuré — Redis n'est qu'un broker ; le throttling de connexion s'exécute en mémoire locale par processus, ce qui perd sa pertinence sous Unicorn multi-processus ;
- **Documentation API** : aucun schéma OpenAPI (drf-spectacular absent) alors que le livrable 4 l'exige ;
- **Rosetta monté sans condition d'environnement** : l'application n'est utile qu'en développement et devrait être retirée de la configuration de production (accès limité au staff, mais surface inutile) ;
- **Recherche** : filtrage « icontains » (LIKE) sur quatre modèles — fonctionnel mais sans index ni pertinence ; la recherche full-text PostgreSQL (SearchVector) est l'évolution naturelle une fois PostGIS actif ;
- **Aucune chaîne CI/CD** : les 391 tests ne sont exécutés nulle part de façon automatique ; pas de lock de dépendances ni de pip-audit ;
- **Fichiers d'export sans expiration** : ExportJob prévoit une durée de vie dans le cahier des charges, absente du modèle — les fichiers générés s'accumulent indéfiniment sur le disque.

5.3 Reproductibilité de l'environnement de test

Note d'audit utile à l'équipe : la suite de tests s'est exécutée sans modification du code grâce à l'option « `SIG_DB_ENGINE=sqlite` » prévue par le projet, sur la bibliothèque SpatiaLite. Ce choix est bon pour la portabilité, mais il implique que la couverture certifiée par l'audit est une couverture SpatiaLite, non PostGIS : les fonctions spécifiques à PostGIS (ST_AsMVT, index GIST avancés, recherche plein texte) ne sont pas exercées par la suite. Lors de l'activation de PostgreSQL (recette puis production), il sera prudent d'exécuter la suite sur PostGIS également — idéalement via un job CI dédié. Par ailleurs, la commande « `makemigrations --check` » ne détecte aucune divergence : les 32 migrations livrées sont complètes et déterministes, condition nécessaire à un premier déploiement serein.

6. Plan d'action priorisé

6.1 Vue d'ensemble des constats

Dix-sept constats ont été consignés, répartis en 2 critiques (chaîne de production et outils d'analyse manquants — tous deux bloquants pour la recette), 8 majeurs (nécessaires à l'exploitation conforme et à la monétisation) et 7 mineurs (qualité, UX et divergences de modèle assumables). La figure 3 illustre cette répartition. Chaque action est estimée en taille d'effort : S (demi-journée au plus), M (un à deux jours), L (trois jours et plus), en développeur confirmé Django/GeoDjango.

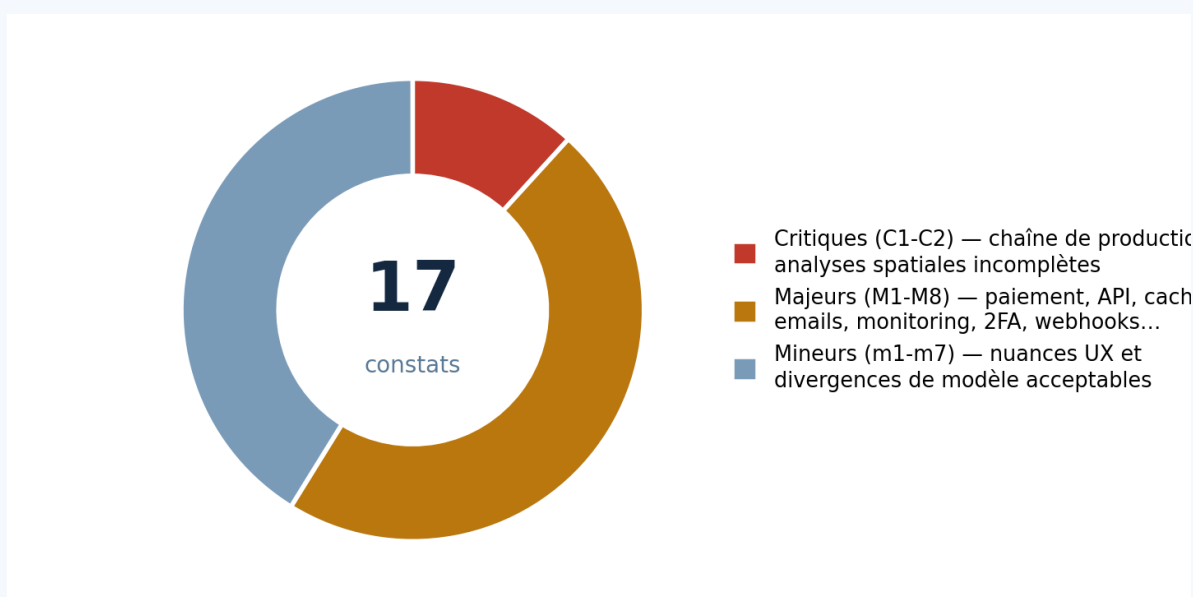


Figure 3 — Répartition des 17 constats par gravité

Tableau 8 — Actions critiques (C1 à C2)

N°	Action	Module	Effort
C1	Construire la chaîne de production exigée : Gunicorn en CMD du Dockerfile (Code 1), service Celery Beat dans le compose, Nginx en reverse proxy, SECRET_KEY obligatoire (retrait du repli « django-insecure-change-me »), procédure de sauvegarde/restauration quotidienne (pg_dump + test de restauration) documentée et testée	devops	L
C2	Compléter le module Analyses spatiales : requête spatiale (intersecte / contient / est dans — primitives GEOS/GeoDjango existantes), calcul de surfaces et périmètres (aire/périmètre en UTM), puis isochrone (décision d'architecture : pgRouting sur PostGIS ou service OSRM/Valhalla à ajouter au compose)	spatial_analysis	L

*Actions critiques — effort cumulé estimé : 7 à 11 jours-homme***Tableau 9 — Actions majeures (M1 à M8)**

N°	Action	Module	Effort
M1	Intégrer le paiement exigé (Stripe ou Flutterwave) : session de paiement et webhook à signature vérifiée, bascule de plan uniquement après paiement, gestion du moyen de paiement et du renouvellement ; modéliser l'entité Subscription (dates, statut) si nécessaire	subscriptions	L
M2	Compléter l'API REST : djangorestframework-gis (sérialisation GeoJSON des géométries), documentation Swagger/ReDoc via drf-spectacular (livrable 4)	api	M
M3	Activer la performance exigée : CACHES Redis (throttling, sessions, données fréquentes), tuiles vectorielles ST_AsMVT avec cache, pagination/limit du GeoJSON, GZip + Whitenoise + ManifestStaticFilesStorage ; benchmarker les KPI (100 000 entités, import 10 000, analyse)	maps / perf	L
M4	Réinitialisation de mot de passe par e-mail + SMTP réel (SendGrid ou SMTP) pour invitations, notifications et resets ; relier le backend e-mail aux réglages par environnement	accounts / email	M
M5	Brancher le centre de notifications sur les événements métier (invitation, import réussi/échoué, export prêt, analyse terminée, partage) et créer l'onglet Notifications des Paramètres (e-mail, push, alertes projets/partages/système)	notifications	M
M6	Intégrer le monitoring exigé : Sentry (erreurs), Prometheus + Grafana (métriques), avec LOGGING Django structuré en production ; statistiques d'usage Matomo en option consentie	observabilité	M
M7	Renforcer l'authentification : 2FA en option (TOTP), exigence de classes de caractères du mot de passe, consultation des sessions actives + déconnexion à distance, parcours de suppression de compte (zone de danger)	accounts	M
M8	Livrer les intégrations manquantes du découpage : application webhooks (événements sortants signés), stockage objet S3/OVH (django-storages) pour imports/exports, et documenter l'alignement du découpage réel (17 apps) avec les 26 apps du CDC	intégrations	M

Actions majeures — effort cumulé estimé : 16 à 23 jours-homme

Tableau 10 — Actions mineures (m1 à m7)

N°	Action	Module	Effort
m1	Import : aperçu des premières lignes avant validation, case « remplacer les données existantes ou créer une couche », progression fine du job	data_import	M
m2	Exports : choix de projection (WGS 84 / Web Mercator / UTM), options avancées (attributs, légende, qualité, filtre spatial) et expiration des fichiers générés	exports	M
m3	Cartes : trancher les statuts (public/privé/partagé/brouillon du CDC vs brouillon/publié/archivé livrés) et implémenter le partage si retenu ; aligner contractuellement	maps	S
m4	Collaborateurs : statut en ligne/hors ligne (dernière activité) et filtre par statut exigés par le CDC	teams	S
m5	Recherche full-text PostgreSQL (SearchVector/trigram) à la place de « icontains », une fois PostGIS actif	search	S
m6	Organization : ajouter e-mail, téléphone et adresse prévus au § 4.4 du CDC (fiche organisation éditable)	organizations	S
m7	Hygiène : conditionner Rosetta à DEBUG, verrouiller les dépendances (lock + pip-audit), CI exécutant les 391 tests sur SQLite et PostGIS	qualité	S

Actions mineures — effort cumulé estimé : 6 à 8 jours-homme

6.2 Séquencement recommandé

Séquence 1 (une à deux semaines) : C1, qui transforme le livrable en plateforme déployable (Gunicorn, Beat, Nginx, sauvegardes, clé obligatoire), accompagné de M4 — sans e-mail transactionnel ni reset de mot de passe, l'exploitation réelle est fragile. Séquence 2 (deux semaines) : C2, en commençant par la requête spatiale et le calcul de surfaces (rapides sur les primitives existantes) avant l'isochrone, qui demande un choix d'architecture (pgRouting ou OSRM) à arbitrer avec le donneur d'ordre. Séquence 3 (deux à trois semaines) : M1 (paiement), M2 (API + Swagger) et M3 (cache, MVT, benchmarks des KPI). Séquence 4 (deux semaines) : M5 à M8 et les actions mineures, avant la recette complète. L'effort total — 29 à 41 jours-homme — reste compatible avec le planning global des 26 semaines, dont les phases « tests et recette » et « déploiement et formation » n'ont pas encore été consommées ; en revanche, la mise en production ne doit pas être prononcée avant l'achèvement des séquences 1 et 2.

7. Conclusion

Le SIG Platform n'est pas un projet à redresser : c'est un projet à **68 % conforme** dont la fondation — la partie la plus difficile à rattraper ensuite — est excellente. Multi-tenant réellement appliqué, rôles contrôlés côté serveur, journal d'audit, quotas incompressibles, CSP stricte sans CDN, 391 tests verts avec 80 % de couverture, configuration de production qui passe « check --deploy » sans un seul avertissement, conteneurisation PostGIS/Redis/Celery fonctionnelle : ces acquis placent la plateforme bien au-dessus de l'état dans lequel se trouvait la vitrine lors de son premier audit, et témoignent d'une équipe qui a intégré les recommandations de la première passe.

Le verdict est « **favorable, sous conditions** » : la recette peut démarrer après les deux actions critiques (chaîne de production C1, outils d'analyse manquants C2), et la mise en production commerciale — celle qui encaisse des paiements et héberge des données clientes — exige en outre les intégrations majeures (paiement M1, e-mails M4,

cache et performance M3, monitoring M6). Aucun de ces chantiers ne remet en cause l'architecture : ils sont périphériques au noyau métier, qui est déjà solide. Avec 29 à 41 jours-homme de remise à niveau et un planning de 26 semaines qui prévoyait justement ces phases, l'équipe dispose de la marge nécessaire pour livrer une plateforme conforme au cahier des charges N°2 — puissante, sécurisée et prête à porter les ambitions de SIG.tg.